

PROJECT REPORT

Automated Deployment System for React Application

An end-to-end CI/CD pipeline built with Git, GitHub, Jenkins, Docker and AWS EC2, with monitoring and alerting through Prometheus and Grafana

Prepared by: M. Akshaya

Repository: Akshaya33-Devops / Automated-React-Deployment (dev branch)

Table of Contents

No.	Section	Page
1	Project Overview	5
1.1	Problem Statement	5
1.2	Scope	5
2	Objective	5
3	Technologies Used	5
3.1	Why This Toolchain	6
4	Project Architecture	6
4.1	Deployment Flow	7
4.2	Monitoring Flow	7
4.3	Component Responsibilities	7
4.4	Repository Structure	8
4.5	Version Control: Git and GitHub	8
5	Docker Configuration	9
5.1	What Docker Is	9
5.2	Image vs Container	9
5.3	Why Nginx	9
5.4	Actual Dockerfile	9
5.5	Local Build and Run	10
5.6	Docker Commands Used	10
6	Jenkins CI/CD Pipeline	10
6.1	CI/CD Theory	10
6.2	Pipeline Stages	11
6.3	Actual Jenkinsfile	12
6.4	Important Lines Explained	13
6.5	Jenkins Credentials	14
6.6	Successful Pipeline Execution	14
7	AWS EC2 Deployment	15
7.1	What EC2 Is	15
7.2	Instance Provisioning	15
7.3	Public and Private IP	16
7.4	Security Group and Ports	16
7.5	Installing Docker on EC2	17
7.6	Deployment Commands	17
7.7	Accessing the Application	18
8	Docker Hub	18
8.1	Repository, Image and Tag	18

No.	Section	Page
8.2	Pushing and Pulling	19
9	Prometheus Monitoring	19
9.1	Node Exporter → Prometheus	19
9.2	Installing and Verifying Node Exporter	20
9.3	Actual prometheus.yml	20
9.4	What the UP Status Means	20
10	Grafana Dashboard	21
10.1	Dashboard and Panels	21
10.2	PromQL Queries Used	21
11	CPU Alert and Email Notification	23
11.1	Alert Rule	23
11.2	How Prometheus and Grafana Work Together	24
11.3	Generating CPU Load for Testing	24
11.4	Gmail Notification Flow	24
12	Testing and Verification	25
12.1	Deployment Verification Checklist	25
13	Security Considerations	26
13.1	Secrets Management	26
13.2	Network Exposure	26
13.3	Container Hygiene	26
14	Troubleshooting	26
14.1	React Build Failed in Jenkins	26
14.2	React Build Failed on AWS EC2	27
14.3	Jenkins SSH Authentication Failed	27
14.4	Docker Container Deployment Conflict	28
14.5	Node Exporter Verification	28
14.6	Prometheus Target Verification	29
14.7	Grafana SMTP Not Configured	29
14.8	Gmail SMTP Authentication Failed	30
14.9	High CPU Alert Stayed in Normal State	30
14.10	Verification Summary	30
14.11	Troubleshooting Commands Reference	31
15	Future Improvements	31
16	Conclusion	31
17	Project Screenshots	
18	Project Links	32

List of Figures

Figure	Description	Section
4.1	System architecture: deployment flow and monitoring flow	4
4.2	GitHub repository structure (react-source/project)	4.4
6.1	Jenkins sign-in page	6.2
6.2	Jenkins job configured with "Pipeline script from SCM"	6.2
6.3	Jenkins credentials store	6.5
6.4	Successful Jenkins pipeline console output	6.6
7.1	EC2 console showing the running instance	7.2
7.2	EC2 security groups	7.4
7.3	Deployed React application at the EC2 public IP	7.7
8.1	Docker Hub repository showing the dev tag	8.2
9.1	Prometheus target health page (node_exporter UP)	9.4
10.1	Grafana dashboard — CPU usage and target status	10.2
10.2	Grafana dashboard — network transmitted and received	10.2
11.1	Grafana High CPU Usage alert rule (firing)	11.2
11.2	Email notification received in Gmail	11.4

1. Project Overview

This report documents an automated deployment system for a React web application. A commit to the dev branch on GitHub triggers a Jenkins pipeline that builds the React app, packages it into a Docker image, pushes that image to Docker Hub, and deploys it to an AWS EC2 instance running Nginx. The running instance is monitored with Node Exporter, Prometheus and Grafana, and Grafana sends an email alert through Gmail SMTP when CPU usage stays high for a sustained period.

The project was built for the GUVI DevOps Engineering certification program to demonstrate hands-on competency across the core DevOps toolchain: version control, containerization, CI/CD, cloud infrastructure and observability. Every stage described here — build, push, deploy, monitor and alert — was implemented and verified before being written up.

1.1 Problem Statement

Manually building, packaging and deploying a web application is slow, repetitive and error-prone once deployments happen more than occasionally. The goal is to remove that manual effort by automating the full path from a code commit to a running, monitored application.

1.2 Scope

The project covers the dev branch workflow: checkout, React build, Docker image build, image push, and deployment to a single EC2 instance, together with monitoring and alerting for that instance. It does not cover a staging/production promotion flow, automated testing, or HTTPS termination — these are listed under Future Improvements.

2. Objective

The objectives of this project are to:

- Automate the build and deployment of a React application using a CI/CD pipeline.
- Package the application as a portable Docker image so it runs identically in any environment.
- Provision and configure an AWS EC2 instance to host the containerized application.
- Set up continuous monitoring of system-level metrics (CPU, memory, disk, network) using Node Exporter and Prometheus.
- Visualize those metrics in Grafana and configure automated email alerting for high CPU usage.
- Document each stage, including the real problems faced and how they were fixed, so the project can be reproduced and reviewed.

3. Technologies Used

The table below lists every tool actually used in this project and the role it plays in the workflow.

Tool	Category	Role in this project
Git & GitHub	Version Control	Hosts the React application source code; the dev branch is what gets checked out and deployed.
Jenkins	CI/CD	Automates checkout, the React build, the Docker image build, the Docker Hub push and the EC2 deployment.
Docker	Containerization	Packages the built React app together with Nginx into a single portable image.
Docker Hub	Image Registry	Stores the built image (akshayanimuthu/react-app, tag dev) so EC2 can pull it independently of Jenkins.
AWS EC2	Cloud Hosting	Runs the Docker container that serves the live application.
Nginx	Web Server	Serves the static React production build from inside the container.
Node Exporter	Metrics Collection	Exposes host-level metrics (CPU, memory, disk, network) on the EC2 instance.
Prometheus	Monitoring	Scrapes and stores the metrics exposed by Node Exporter.
Grafana	Visualization / Alerting	Reads metrics from Prometheus, renders dashboards, and fires an email alert via Gmail SMTP.

3.1 Why This Toolchain

Each tool maps to a specific stage of the deployment lifecycle: Git/GitHub for source control, Jenkins for pipeline orchestration, Docker for consistent packaging, Docker Hub for image distribution, AWS EC2 for compute, and the Prometheus/Grafana stack for observability. Together they form a compact but realistic setup that mirrors what DevOps and SRE teams use in production.

4. Project Architecture

The architecture connects five logical layers: source control, CI/CD orchestration, image registry, compute/runtime, and observability. The diagram below shows how a code change moves through the system and how the resulting server is monitored.

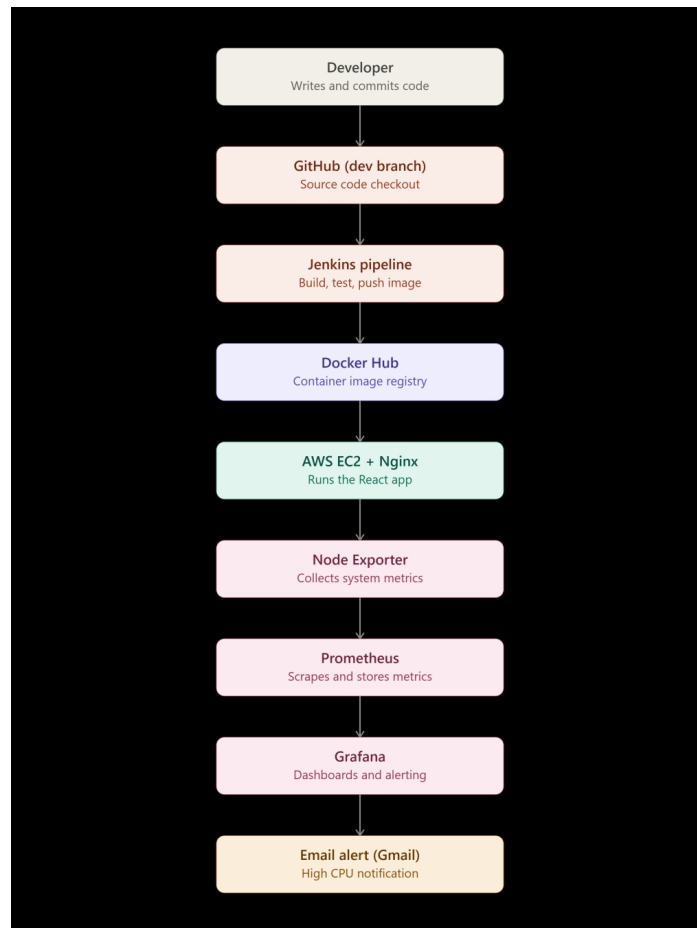


Figure 4.1 — System architecture: deployment flow and monitoring flow.

4.1 Deployment Flow

Developer → GitHub (dev branch) → Jenkins → React Build → Docker Build → Docker Hub → AWS EC2 → Nginx → React Application

4.2 Monitoring Flow

AWS EC2 → Node Exporter → Prometheus → Grafana → CPU Alert → Gmail Notification

4.3 Component Responsibilities

Layer	Responsibility
Source Control	GitHub stores the application code; commits on the dev branch are what Jenkins checks out for a new deployment.
CI/CD Orchestration	Jenkins checks out the code, builds the React application, builds the Docker image, and drives the deployment steps.
Image Registry	Docker Hub stores the versioned image (akshayanimuthu/react-app:dev) so EC2 can pull it independently of Jenkins.
Compute & Runtime	The EC2 instance runs the Docker container; Nginx inside the container serves the built React app on port 80.
Observability	Node Exporter, Prometheus and Grafana collect, store, visualize and alert on system metrics from the EC2 host.

Pulling the image from Docker Hub on the EC2 host, rather than copying files directly from Jenkins, keeps the deployment step simple and repeatable: the same pull-stop-run sequence works whether it is triggered by Jenkins or run manually over SSH. It also keeps the build environment separate from the runtime environment.

4.4 Repository Structure

The actual repository layout, confirmed from the GitHub repository view, is as follows:

```
Automated-React-Deployment/      (dev branch)
├── build/                      # Committed production build snapshot
├── react-source/
│   └── project/
│       ├── public/
│       ├── src/
│       ├── Dockerfile          # Packages the build output with Nginx
│       └── package.json        # React dependencies and build scripts
├── screenshots/               # Evidence screenshots referenced in this report
├── Architecture.png            # Architecture diagram source
├── Jenkinsfile                 # Actual CI/CD pipeline definition
├── Project Report.pdf
├── README.md
└── Troubleshooting Document.pdf
```

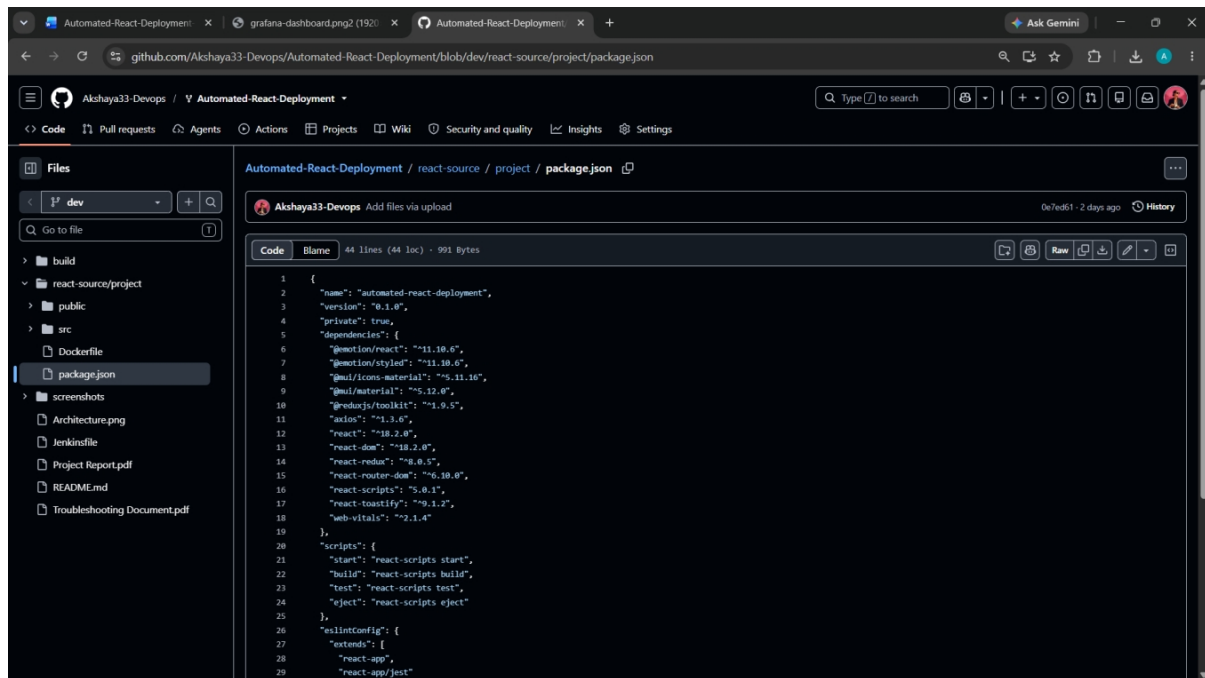


Figure 4.2 — GitHub repository view confirming the folder structure (react-source/project/package.json shown).

The React source, package.json and the Dockerfile all live inside react-source/project/, not at the repository root. The Jenkins pipeline therefore changes into that directory before running the npm build and the Docker build, which is reflected in the actual Jenkinsfile in Section 6.3.

4.5 Version Control: Git and GitHub

Git is the distributed version control system used to track changes to the React application source code. GitHub hosts that repository online (Akshaya33-Devops/Automated-React-Deployment) and gives Jenkins a URL to clone from.

All development work happens on the dev branch rather than main, so that ongoing changes could be committed and tested through the pipeline without a separate release/promotion process. Code reaches Jenkins through the pipeline's Checkout Code stage, which clones the dev branch directly using Jenkins' built-in Git integration — no manual copying of files is involved, so every run starts from whatever is currently committed to dev.

5. Docker Configuration

5.1 What Docker Is

Docker is a containerization platform. It packages an application together with everything it needs to run — runtime, libraries, configuration — into a single portable unit. Because the container carries its own environment, the same artifact behaves identically on a developer laptop, on the Jenkins agent, and on the EC2 server, which removes the "works on my machine" class of deployment failure.

5.2 Image vs Container

Concept	What it is	In this project
Image	A read-only, layered template built from a Dockerfile. It is immutable and can be versioned, stored in a registry and copied between machines.	akshayamanimuthu/react-app:dev — the React production build plus Nginx.
Container	A running instance of an image, with its own writable layer, process and network namespace. Many containers can run from one image.	react-container — the single container running on EC2 with port 80 published.

In short: the image is the recipe and the container is the meal. Building produces an image; running produces a container.

5.3 Why Nginx

The React build produces a set of static files (HTML, JS, CSS) — it does not need a Node.js server in production. Nginx is a lightweight, high-performance web server well suited to serving static files, so the image ships Nginx rather than the Node.js toolchain.

5.4 Actual Dockerfile

The Dockerfile used in this project, located at `react-source/project/Dockerfile`, is:

```
FROM nginx:alpine
COPY build /usr/share/nginx/html
EXPOSE 80
```

Line-by-line explanation:

- `FROM nginx:alpine` — starts from the lightweight Alpine Linux build of the official Nginx image, which keeps the final image small and reduces the attack surface.
- `COPY build /usr/share/nginx/html` — copies the React production build output into Nginx's default web root, replacing Nginx's default landing page with the React app.
- `EXPOSE 80` — documents that the container listens on port 80. The actual host-to-container port mapping is set at run time with the `-p` flag.

5.5 Local Build and Run

Before involving Jenkins or EC2, the image was built and run locally to validate the Dockerfile:

```
docker build -t react-app .
docker run -d -p 8080:80 --name react-container react-app
```

The build command reads the Dockerfile in the current directory and tags the resulting image as `react-app`. The run command starts a container from that image in the background (`-d`), maps host port 8080 to the container's port 80, and names the container `react-container`. The application was then verified at `http://localhost:8080` before moving on to the CI/CD and cloud stages.

5.6 Docker Commands Used

These are the commands actually used throughout development and troubleshooting:

```

docker ps
docker images
docker stop react-container
docker start react-container
docker logs --tail 50 react-container
docker rm react-container
docker update --restart unless-stopped react-container

```

Command	Purpose
docker ps	Lists running containers.
docker images	Lists locally available images.
docker stop / start	Stops or starts a named container.
docker logs --tail 50	Shows the last 50 log lines from a container — used to diagnose start-up issues.
docker rm	Removes a stopped container so a new one can be created with the same name.
docker update --restart unless-stopped	Changes an existing container's restart policy without recreating it.

6. Jenkins CI/CD Pipeline

6.1 CI/CD Theory

Continuous Integration (CI) means every change committed to the branch is automatically built and validated, so integration problems surface within minutes of the commit rather than at the end of a release cycle. Continuous Deployment (CD) extends that: once a build succeeds, it is shipped to the target environment automatically, with no manual steps. Together they turn deployment from an event into a routine, repeatable operation.

Jenkins is an open-source automation server that runs these pipelines. It was chosen because it is one of the most widely used CI/CD tools in industry and integrates directly with GitHub, Docker and SSH-based deployment. The pipeline here is written as a declarative Jenkinsfile committed to the repository, so the pipeline definition is itself version-controlled alongside the application.

6.2 Pipeline Stages

Checkout Code → Build React App → Build Docker Image → Push to Docker Hub → Deploy to EC2 (pull image → stop old container → run new container)

Stage	Action	Purpose
Checkout Code	Clones the dev branch from GitHub.	Ensures the build always starts from the latest committed source.
Build React App	Runs npm install and npm run build inside react-source/project, with CI unset and the ESLint plugin disabled.	Produces the static production files that the Docker image will package.
Build Docker Image	Runs docker build using the Dockerfile in react-source/project.	Produces a reproducible, versioned application image.

Stage	Action	Purpose
Push to Docker Hub	Logs in with the stored credentials and pushes the image tagged dev.	Makes the image available for the EC2 host to pull.
Deploy to EC2	Connects over SSH, pulls the image, stops/removes the old container, and starts the new one.	Rolls out the new version with minimal downtime.

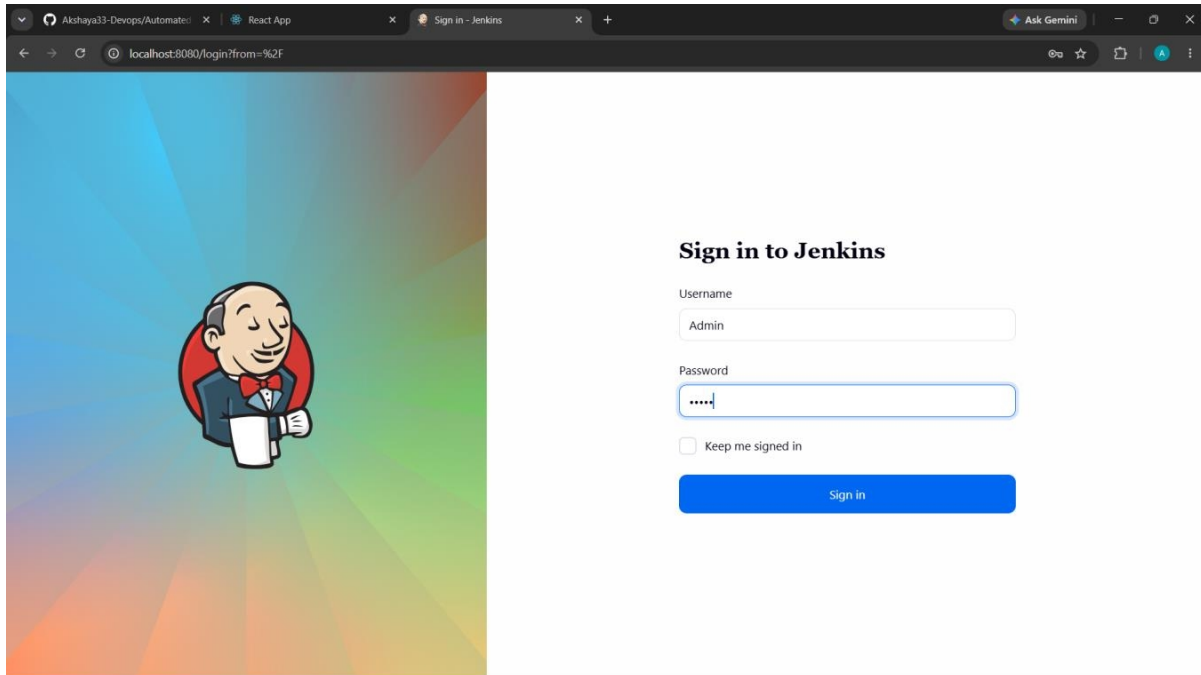


Figure 6.1 — Jenkins sign-in page for the local Jenkins controller.

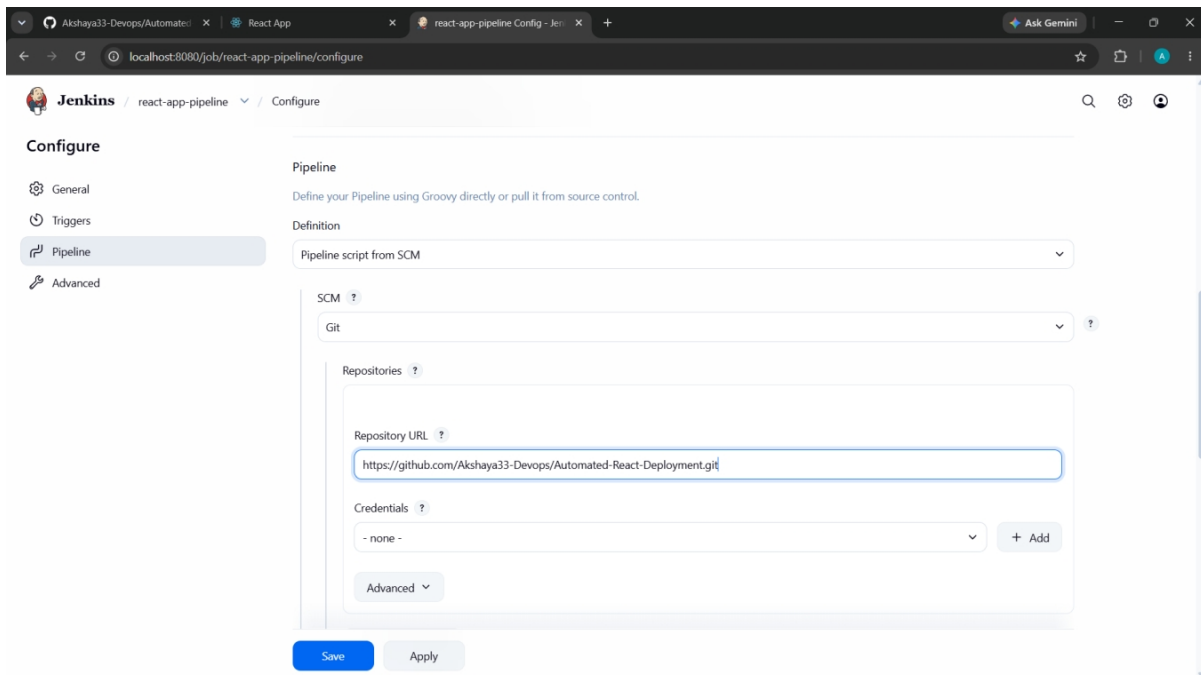


Figure 6.2 — Jenkins job configured with "Pipeline script from SCM", pointed at the GitHub repository.

6.3 Actual Jenkinsfile

The declarative pipeline below is the actual Jenkinsfile used in this project. It includes the Build React App stage with the exact build command that had to be fixed during development; see Troubleshooting 14.1 for why the CI and DISABLE_ESLINT_PLUGIN flags are set.

```
pipeline {
  agent any

  environment {
    DOCKERHUB_CREDENTIALS = credentials('dockerhub-credentials')
    IMAGE_NAME = 'akshayanimuthu/react-app'
    IMAGE_TAG = 'dev'
  }

  stages {
    stage('Checkout Code') {
      steps {
        git branch: 'dev',
            url: 'https://github.com/Akshaya33-Devops/Automated-React-Deployment.git'
      }
    }

    stage('Build React App') {
      steps {
        dir('react-source/project') {
          bat 'npm install'
          bat 'set "CI=" && set "DISABLE_ESLINT_PLUGIN=true" && npm run build'
        }
      }
    }

    stage('Build Docker Image') {
      steps {
        dir('react-source/project') {
          sh 'docker build -t $IMAGE_NAME:$IMAGE_TAG .'
        }
      }
    }

    stage('Push to Docker Hub') {
      steps {
        sh 'echo $DOCKERHUB_CREDENTIALS_PSW | docker login \
          -u $DOCKERHUB_CREDENTIALS_USR --password-stdin'
        sh 'docker push $IMAGE_NAME:$IMAGE_TAG'
      }
    }

    stage('Deploy to EC2') {
      steps {
        sshagent(['ec2-ssh-key']) {
          sh '''
            ssh -o StrictHostKeyChecking=no ec2-user@13.201.83.88 "
              docker pull $IMAGE_NAME:$IMAGE_TAG &&
              docker stop react-container || true &&
              docker rm react-container || true &&
              docker run -d -p 80:80 --name react-container \
                --restart unless-stopped $IMAGE_NAME:$IMAGE_TAG
            "
          '''
        }
      }
    }
  }
}
```

```
}  
}
```

6.4 Important Lines Explained

- `credentials('dockerhub-credentials')` — loads the stored Docker Hub username and password into environment variables without ever printing them in plain text.
- `dir('react-source/project')` — changes into the folder that actually contains `package.json` and the `Dockerfile`, matching the real repository layout in Figure 4.2.
- `bat 'set "CI=" && ... npm run build'` — runs the React production build on the Windows Jenkins agent; `CI` is cleared and the ESLint plugin is disabled so warnings do not fail the build.
- `docker login --password-stdin` — logs in to Docker Hub by piping the password through standard input instead of passing it as a visible command-line argument.
- `sshagent(['ec2-ssh-key'])` — loads the stored EC2 private key for the duration of the block so the `ssh` command inside it can authenticate.
- `docker stop ... || true` and `docker rm ... || true` — remove any existing container without failing the pipeline when no such container exists yet.
- `docker run -d ... --restart unless-stopped` — starts the new container in the background and configures it to restart automatically if the instance reboots or the container crashes.

6.5 Jenkins Credentials

Secrets are never written into the Jenkinsfile. Both are stored in Jenkins' built-in credentials store and referenced by ID, so the pipeline definition can live in a public repository safely.

Credential ID	Type	Purpose
dockerhub-credentials	Username with password	Authenticates and pushes images to Docker Hub.
ec2-ssh-key	SSH username with private key	Used by Jenkins to connect to the EC2 instance for deployment (user: <code>ec2-user</code>).

Neither credential is written to the repository, the Jenkinsfile, or any console output in plain text — Jenkins masks the values in the build log.

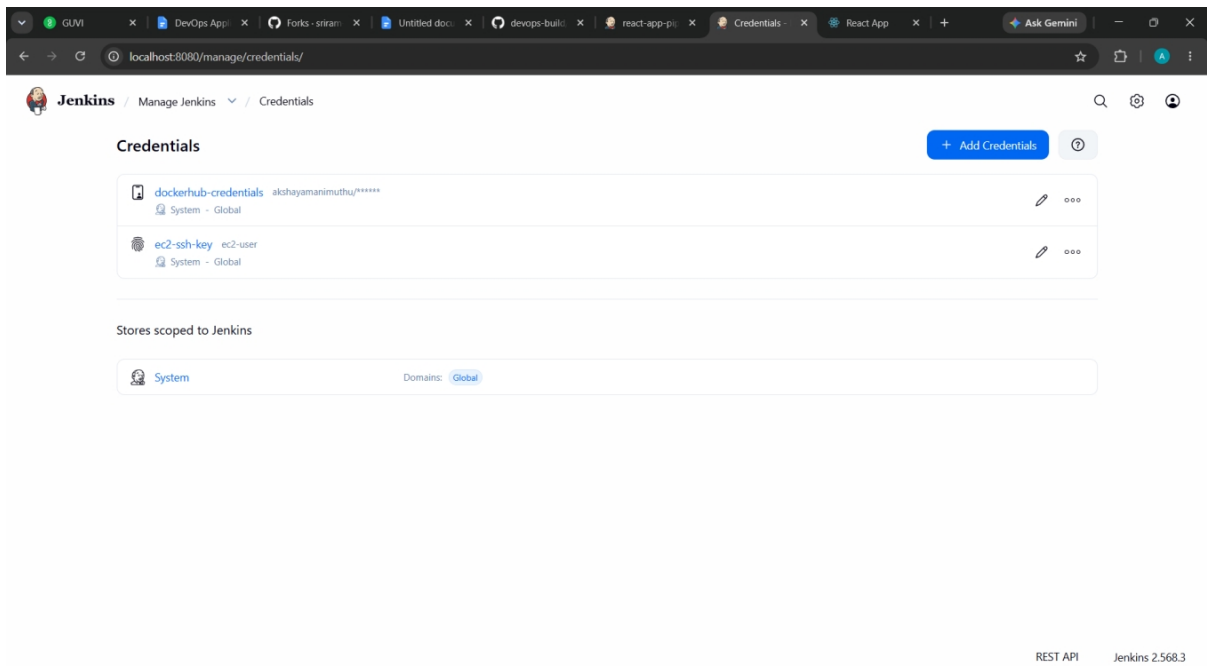


Figure 6.3 — Jenkins credentials store showing dockerhub-credentials and ec2-ssh-key.

6.6 Successful Pipeline Execution

The console output below is from a completed run. It shows the deploy stage pulling the newer image, stopping and removing the previous react-container, starting the new one, and the pipeline finishing with a SUCCESS status.

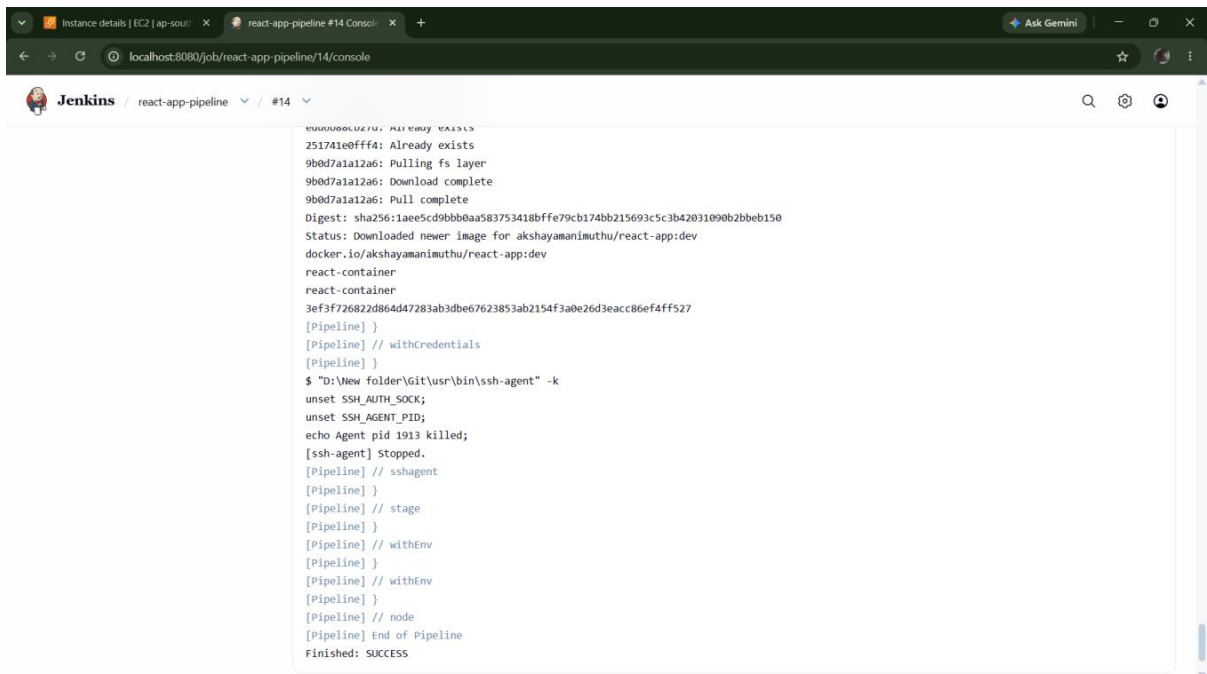


Figure 6.4 — Console output of a successful Jenkins pipeline run (Finished: SUCCESS).

7. AWS EC2 Deployment

7.1 What EC2 Is

Amazon EC2 (Elastic Compute Cloud) provides virtual servers, called instances, that are rented by the hour from AWS. An instance is launched from an AMI (a machine image defining the operating system), sized by an instance type (how much CPU and memory it gets), placed in a region and availability zone, and protected by a security group that acts as a virtual firewall. Access is by SSH key pair rather than a password.

EC2 was used here to host the Docker container running the React application, giving it a public, always-on address without managing physical hardware.

7.2 Instance Provisioning

Setting	Value
Operating System	Amazon Linux 2023
Instance Type	t3.micro
Public IP	13.201.83.88
Private IP	172.31.11.59
Region	Asia Pacific (Mumbai) — ap-south-1

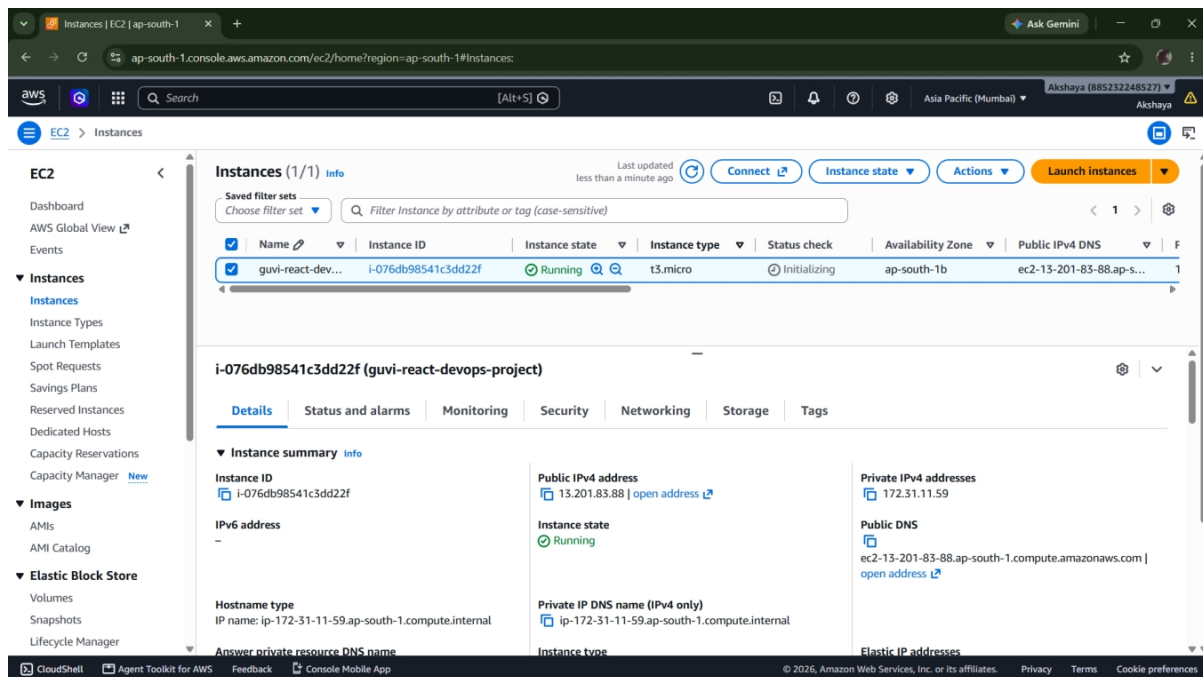


Figure 7.1 — EC2 console showing the running instance (t3.micro, public IP 13.201.83.88).

7.3 Public and Private IP

The private IP (172.31.11.59) is only reachable from within the same AWS VPC and is used for internal networking. The public IP (13.201.83.88) is reachable from the internet and is how the React application, Prometheus and Grafana are accessed from outside AWS.

7.4 Security Group and Ports

A security group is a stateful, allow-only firewall attached to the instance: any inbound port not explicitly opened is dropped, and return traffic for an allowed connection is permitted automatically. Only the ports this system actually needs are opened.

Port	Service	Purpose
22	SSH	Used by Jenkins (and manually) to connect to the instance for deployment and administration.
80	HTTP	Serves the React application through Nginx running inside the Docker container.
3000	Grafana	Grafana's web UI, used for dashboards and alert configuration.
9090	Prometheus	Prometheus' web UI, used to check target health and run queries.
9100	Node Exporter	The metrics endpoint that Prometheus scrapes for host-level metrics.

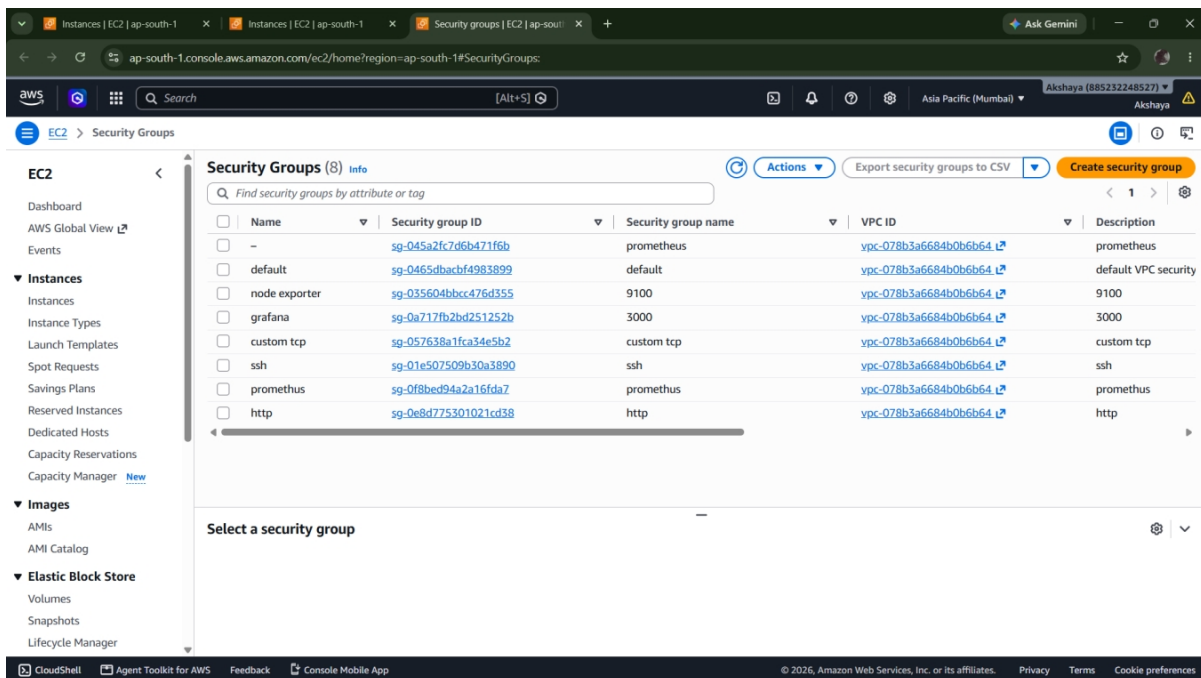


Figure 7.2 — EC2 security groups, one per service (ssh, http, grafana, node exporter, prometheus).

Each service has its own dedicated security group rather than one combined rule set, which is why the console lists separate groups named ssh, http, grafana, node exporter and prometheus.

7.5 Installing Docker on EC2

```
sudo dnf update -y
sudo dnf install docker -y
sudo systemctl start docker
sudo systemctl enable docker
sudo usermod -aG docker ec2-user
```

- `dnf update / install docker` — updates the package index and installs the Docker engine using Amazon Linux's package manager.

- `systemctl start / enable docker` — starts the Docker service immediately and configures it to start on every boot.
- `usermod -aG docker ec2-user` — adds `ec2-user` to the `docker` group so Docker commands run without `sudo`. The SSH session must be restarted for the new group membership to take effect; this is required for the deployment commands Jenkins runs over SSH.

7.6 Deployment Commands

These are the exact commands the Deploy to EC2 stage runs on the instance, and the same sequence used when deploying manually over SSH:

```
docker pull akshayanimuthu/react-app:dev

docker stop react-container || true
docker rm react-container || true

docker run -d \
  -p 80:80 \
  --name react-container \
  --restart unless-stopped \
  akshayanimuthu/react-app:dev
```

The `--restart unless-stopped` flag ensures the container automatically restarts after a host reboot or an unexpected crash, without anyone logging in to start it manually.

7.7 Accessing the Application

Once the container is running, the application is publicly reachable at `http://13.201.83.88`, served directly by the Nginx process inside the container on port 80.

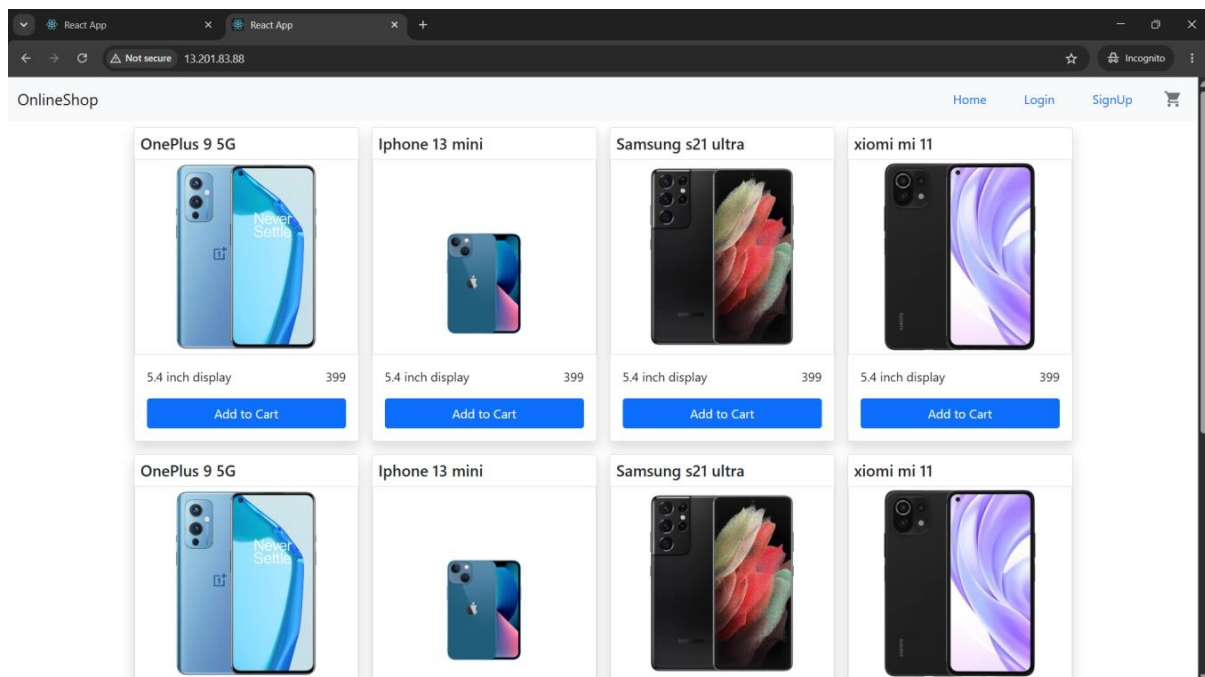


Figure 7.3 — The deployed React application ("OnlineShop") reachable at the EC2 public IP.

8. Docker Hub

Docker Hub is a cloud-based registry for Docker images — a place to store built images so they can be pulled onto any other machine without rebuilding from source. In this project it is the hand-off point between Jenkins (which builds the image) and the EC2 instance (which runs it).

8.1 Repository, Image and Tag

Term	Value here	Meaning
Repository	akshayanimuthu/react-app	The named location on Docker Hub where every version of this image is stored. The prefix is the Docker Hub account name.
Image	React production build + Nginx	The packaged artifact built from the Dockerfile in Section 5.4.
Tag	dev	Identifies this build as the one coming from the dev branch. Every pipeline run pushes to the same dev tag, so it always points at the most recently built image.

8.2 Pushing and Pulling

```
docker login -u akshayanimuthu
docker push akshayanimuthu/react-app:dev
docker pull akshayanimuthu/react-app:dev
```

Jenkins pushes the freshly built image at the end of the Push to Docker Hub stage. The EC2 instance then independently pulls that same image during the Deploy to EC2 stage — the EC2 host never receives files directly from Jenkins, it always goes through Docker Hub.

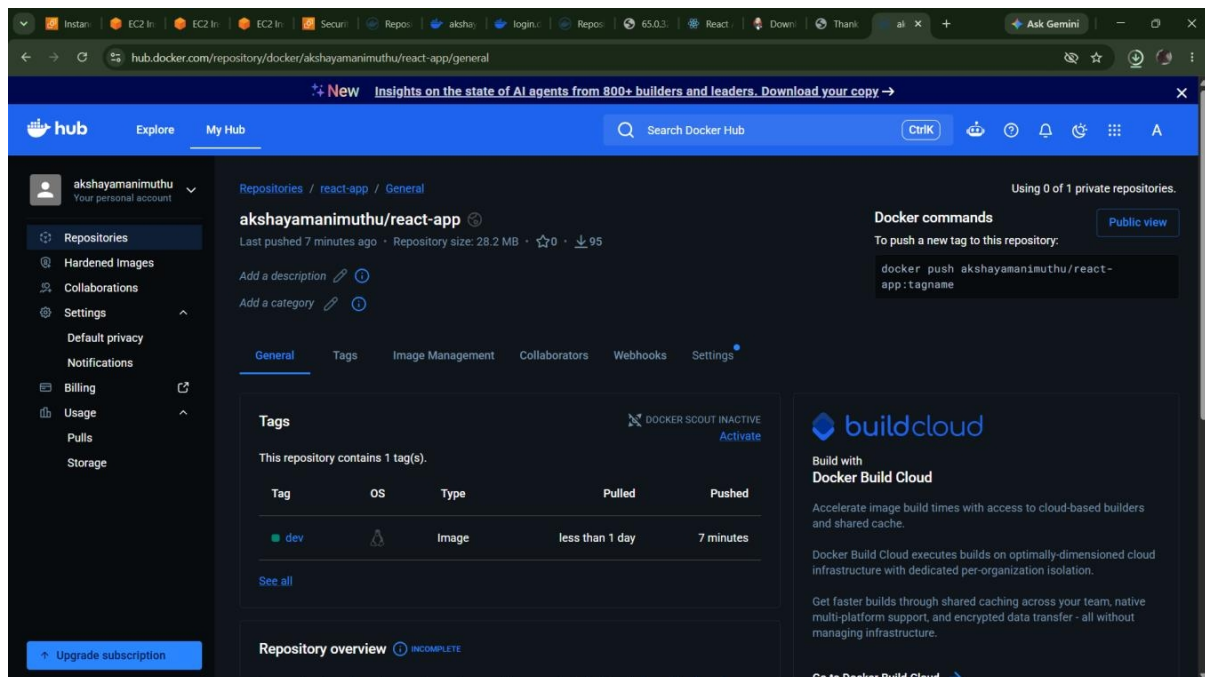


Figure 8.1 — Docker Hub repository akshayanimuthu/react-app showing the dev tag.

9. Prometheus Monitoring

Monitoring means continuously collecting measurements about a running system — CPU, memory, disk, network, application health — so problems can be noticed and understood instead of being discovered through user complaints. This project adds monitoring on top of the deployment pipeline using Node Exporter, Prometheus and Grafana.

9.1 Node Exporter → Prometheus

Node Exporter is a small agent that runs on the EC2 instance and reads host-level statistics from the Linux kernel (`/proc` and `/sys`), publishing them as plain text over HTTP on port 9100 at the `/metrics` path. It only exposes data; it stores nothing.

Prometheus is the time-series database that does the storing. It works on a pull model: at a fixed interval it "scrapes" the `/metrics` endpoint, timestamps every value it finds, and writes it to its local time-series store, where it can be queried later with PromQL. Each endpoint Prometheus scrapes is called a target. So the chain is: kernel → Node Exporter (exposes) → Prometheus (scrapes and stores) → Grafana (queries and displays).

9.2 Installing and Verifying Node Exporter

```
sudo systemctl daemon-reload
sudo systemctl enable --now node_exporter
sudo systemctl status node_exporter
curl http://localhost:9100/metrics | head
```

- `systemctl enable --now node_exporter` — enables the service to start on boot and starts it immediately in one command.
- `curl http://localhost:9100/metrics | head` — confirms the exporter is actually serving metrics text on port 9100.

9.3 Actual prometheus.yml

The scrape configuration used in this project is:

```
scrape_configs:
- job_name: "node_exporter"
  static_configs:
  - targets: ["localhost:9100"]
```

- `job_name: "node_exporter"` — labels every metric collected from this target with `job="node_exporter"`, which is how Grafana later filters for these metrics.
- `targets: ["localhost:9100"]` — tells Prometheus exactly where to scrape from; since Prometheus and Node Exporter run on the same EC2 instance, this is localhost on port 9100.

The configuration was validated before relying on it, and Prometheus readiness was checked:

```
sudo /opt/prometheus/promtool check config /etc/prometheus/prometheus.yml
curl http://localhost:9090/-/ready
```

9.4 What the UP Status Means

Prometheus records the outcome of every scrape as a synthetic metric called `up`. A value of 1 (shown as UP in the UI) means the last scrape of that target succeeded; 0 (DOWN) means it failed — the exporter was not

running, the port was blocked by the security group, or the address was wrong. Checking the Status → Target health page is therefore the quickest way to confirm the whole collection chain is working before building any dashboard on top of it.

In this project the `node_exporter` target is shown with a state of UP, confirming metrics are being collected successfully.

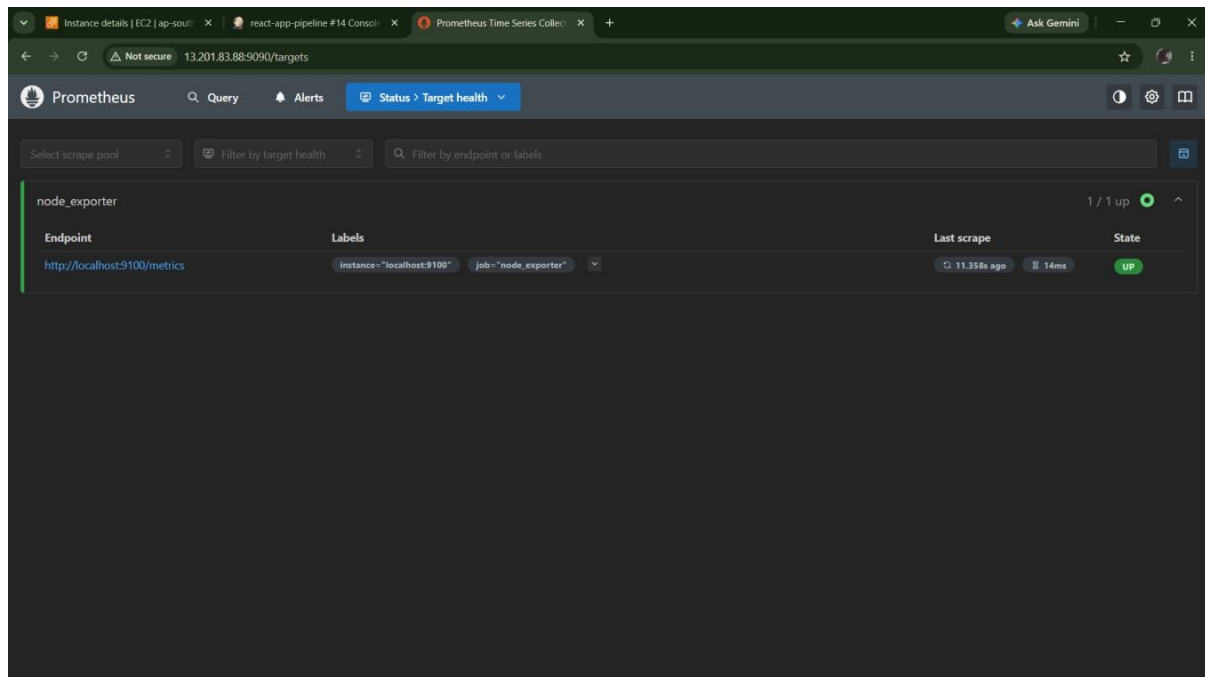


Figure 9.1 — Prometheus target health page showing the `node_exporter` target as UP (13.201.83.88:9090/targets).

10. Grafana Dashboard

Grafana is a visualization and alerting tool. It does not collect metrics itself — it connects to a metrics source (a "data source"), turns raw numbers into readable graphs, and can watch those numbers and fire alerts. Grafana was connected to Prometheus at `http://localhost:9090` as its data source, because Prometheus is where all of the EC2 instance's metrics are already stored.

10.1 Dashboard and Panels

A dashboard is a saved collection of panels; each panel is one query rendered as a graph, gauge or stat. A dashboard named EC2 Monitoring was built with the following panels:

- Target Status — confirms Prometheus is actively scraping the Node Exporter target.
- CPU Usage — percentage CPU utilization over time.
- Memory Usage — used vs. available memory.
- Disk Usage — storage utilization on the root volume.
- Network Transmitted — outbound network traffic, broken down by interface (docker0, ens5, veth...).
- Network Received — inbound network traffic, broken down by the same interfaces.

10.2 PromQL Queries Used

PromQL is Prometheus' query language. The queries behind each panel are:

Target Status

```
up{instance="localhost:9100", job="node_exporter"}
```

CPU Usage (%)

```
100 - (avg(rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100)
```

Memory Usage (%)

```
(1 - (node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes)) * 100
```

Disk Usage (%) — root volume

```
100 - ((node_filesystem_avail_bytes{mountpoint="/"}/ node_filesystem_size_bytes{mountpoint="/"}) * 100)
```

Network Transmitted / Received (bytes per second)

```
rate(node_network_transmit_bytes_total[5m])  
rate(node_network_receive_bytes_total[5m])
```

A note on `rate()`: counters such as `node_cpu_seconds_total` only ever increase, so their raw value is meaningless on a graph. `rate()` converts a counter into a per-second average over the given window — here 5 minutes — which is what makes CPU and network usable as live utilization panels. The CPU query subtracts idle time from 100 because Node Exporter reports time spent in each CPU mode rather than a ready-made utilization percentage.

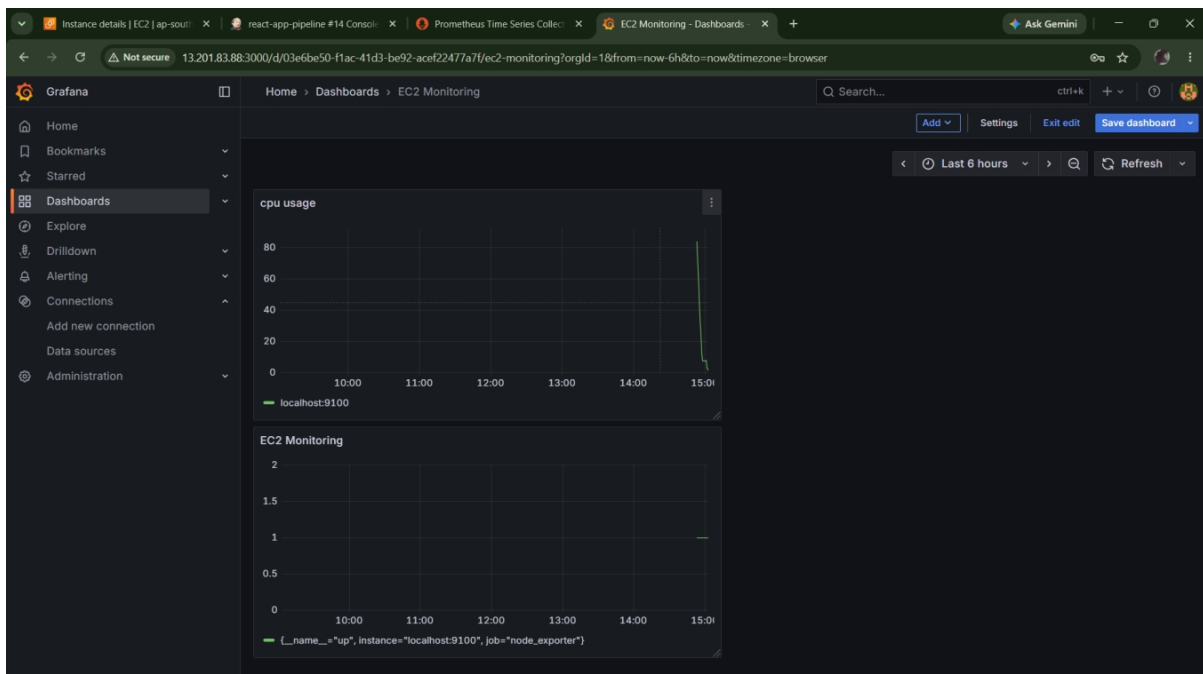


Figure 10.1 — Grafana EC2 Monitoring dashboard: CPU usage and target status panels.

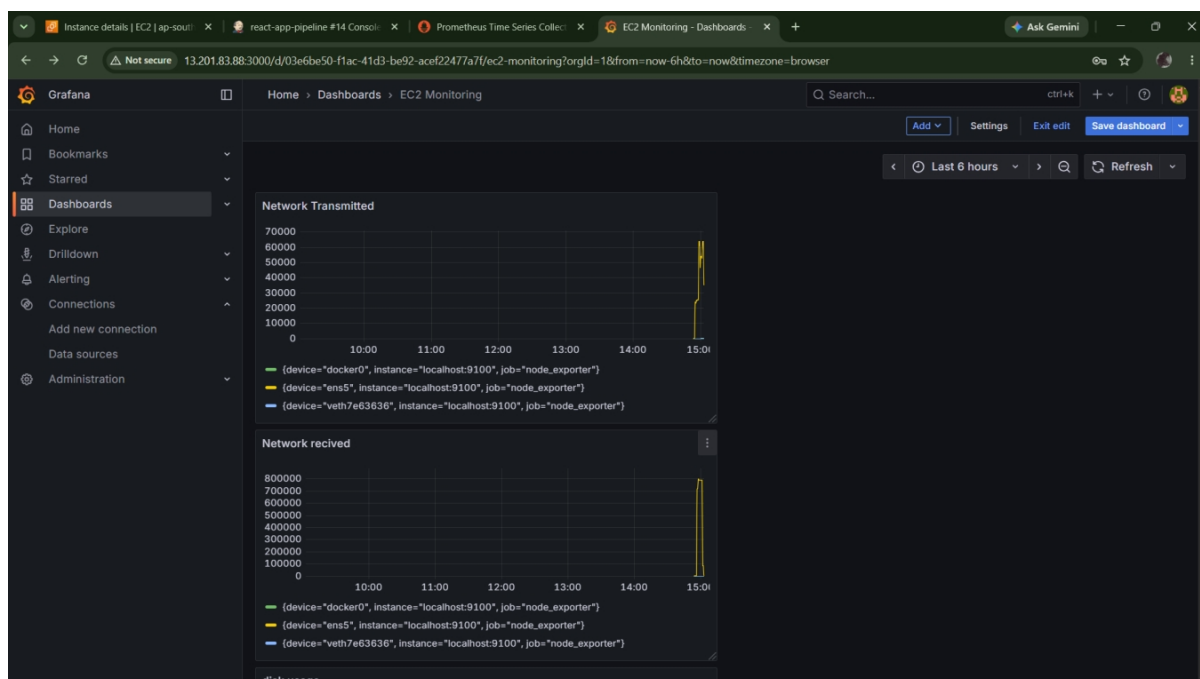


Figure 10.2 — Grafana EC2 Monitoring dashboard: network transmitted and received panels.

Together these panels give a single view of the health of the EC2 instance hosting the application, making it easier to spot resource pressure before it affects users.

11. CPU Alert and Email Notification

A dashboard still requires someone to remember to look at it. To move to a system that actively notifies someone, a Grafana alert rule was configured to watch CPU usage and send an email when it stays high for a sustained period.

11.1 Alert Rule

A Grafana alert rule has three parts: a query (which metric to evaluate), a condition and threshold (when it counts as a problem), and a contact point (where the notification goes). The rule is evaluated on a fixed interval and moves through states — Normal, Pending, Firing — before notifying.

Setting	Value
Alert Name	High CPU Usage
Query	$100 - (\text{avg}(\text{rate}(\text{node_cpu_seconds_total}\{\text{mode}=\text{"idle"}\}[5\text{m}])) * 100$
Condition	IS ABOVE 80
CPU Threshold	80%
Pending Period	5 minutes
Contact Point	Email (Gmail SMTP)

The pending period exists so that a brief, momentary CPU spike does not immediately raise an alarm. CPU usage must stay above 80% continuously for the full five minutes before Grafana transitions the alert from

Pending to Firing and sends the notification. This avoids noisy false alerts for short-lived spikes while still catching genuinely sustained load.

11.2 How Prometheus and Grafana Work Together

Prometheus continuously scrapes and stores the CPU metric from Node Exporter. Grafana's alerting engine periodically evaluates the same metric against the configured rule. When the condition has been true for the entire pending period, Grafana sets the alert state to Firing and hands the notification off to the configured contact point — in this case a Gmail SMTP email.

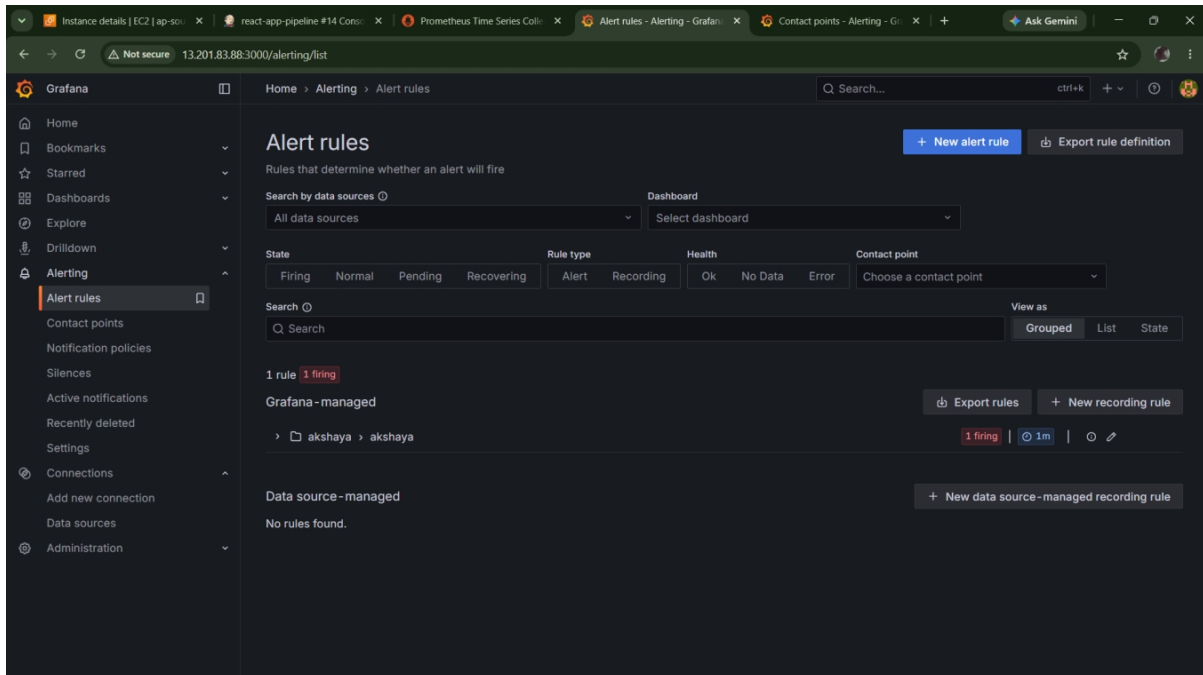


Figure 11.1 — Grafana alert rules page showing the High CPU Usage alert in a firing state.

11.3 Generating CPU Load for Testing

An alert that has never fired is an untested alert. Under normal conditions the t3.micro instance sits far below 80% CPU, so load was generated deliberately on the EC2 instance to verify the rule end-to-end:

```
stress-ng --cpu 2 --timeout 5m
```

This runs CPU-bound work on two workers for five minutes — long enough to push utilization above the 80% threshold for the full pending period and confirm the alert transitions to Firing. After testing, the threshold was restored to its final value of 80% with a 5-minute pending period.

11.4 Gmail Notification Flow

The notification path is: Grafana alert fires → Grafana connects to smtp.gmail.com on port 587 → StartTLS upgrades the connection to encrypted → Grafana authenticates with the Gmail address and a Google App Password → Gmail relays the message → the email arrives in the inbox. The relevant section of grafana.ini is shown below with placeholder values; the actual Gmail address and App Password are deliberately not reproduced in this report.

```
[smtp]
```

```

enabled = true
host = smtp.gmail.com:587
user = YOUR_GMAIL_ADDRESS
password = YOUR_GOOGLE_APP_PASSWORD
from_address = YOUR_GMAIL_ADDRESS
from_name = Grafana
startTLS_policy = MandatoryStartTLS

```

A Google App Password is used instead of the normal account password because Gmail does not accept regular account passwords for third-party SMTP logins. Grafana must be restarted after changing grafana.ini for the settings to take effect.

```
sudo systemctl restart grafana-server
```

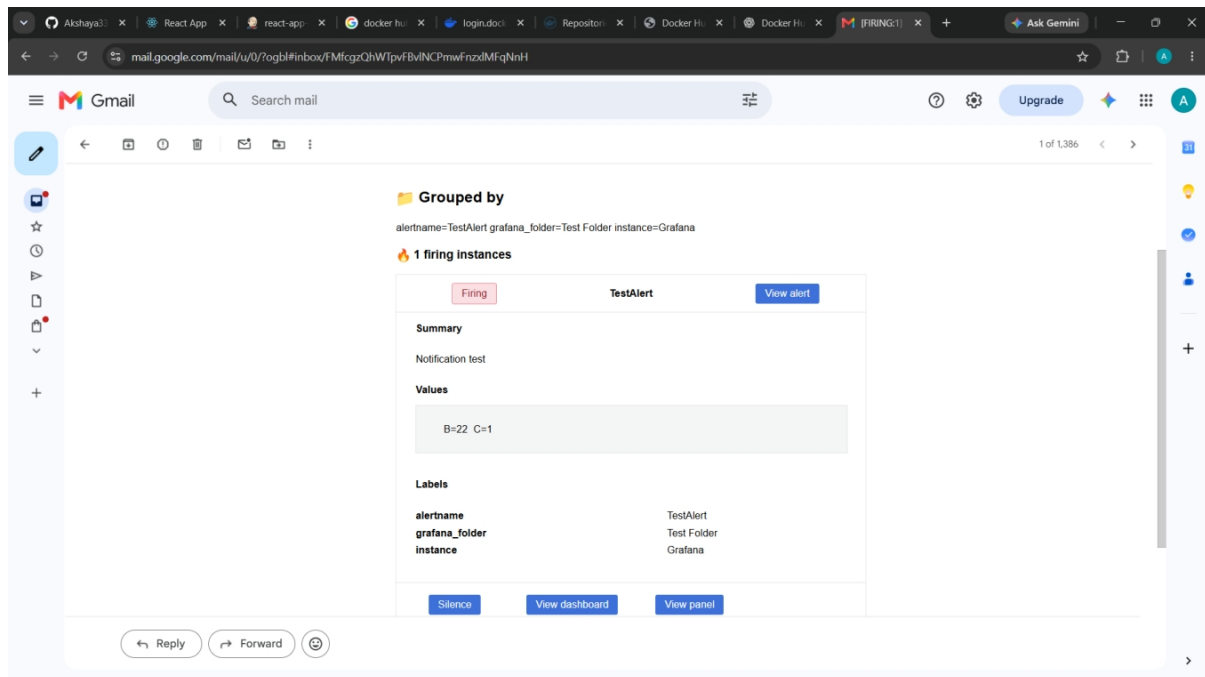


Figure 11.2 — Email notification received in Gmail when the High CPU Usage alert fired.

12. Testing and Verification

Each stage of the pipeline was validated on its own before being chained into the full automated workflow, and the end-to-end flow was then re-verified as a whole.

- Built and ran the Docker image locally, confirming the React app was served correctly on port 8080.
- Triggered the Jenkins pipeline and confirmed every stage (checkout, React build, Docker build, push, deploy) completed without errors.
- Confirmed the image appeared in the Docker Hub repository under the dev tag after a successful push.
- Verified the container was running on EC2 with `docker ps` and confirmed the app loaded at the public IP.
- Checked the Prometheus targets page to confirm the Node Exporter target reported as UP.
- Opened the Grafana dashboard and confirmed all panels displayed live data.
- Generated CPU load with `stress-ng` and confirmed the Grafana alert fired and the email notification was received.

12.1 Deployment Verification Checklist

Item	Status
React application built and tested locally	Verified
Docker image built successfully	Verified
Docker image pushed to Docker Hub (dev tag)	Verified
Docker installed and running on EC2	Verified
React container running on EC2	Verified
Application reachable through the EC2 public IP	Verified
Jenkins pipeline completed successfully	Verified
Node Exporter installed and exposing metrics	Verified
Prometheus target verified as UP	Verified
Grafana dashboard connected to Prometheus	Verified
High CPU alert configured and tested	Verified
Email notification received via Gmail	Verified

13. Security Considerations

13.1 Secrets Management

- Docker Hub credentials and the EC2 SSH private key are stored only inside Jenkins' credentials store, never in the repository.
- Private SSH keys, AWS access keys, Docker Hub passwords/tokens, .pem files and the Gmail App Password are never committed to version control or written into this report.

13.2 Network Exposure

- The EC2 security groups only open the ports the system actually needs: 22, 80, 3000, 9090 and 9100.
- Restricting SSH (port 22) to specific trusted IP ranges, rather than leaving it open to 0.0.0.0/0, is listed as a future hardening step.

13.3 Container Hygiene

- The nginx:alpine base image keeps the runtime footprint small, reducing the attack surface compared with a full OS-based image.
- The --restart unless-stopped policy keeps the service self-healing without needing an externally exposed management interface.

14. Troubleshooting

This section documents the real problems encountered while building this project. Each follows the same approach: identify the symptom, check logs and configuration, find the root cause, apply the fix, rerun, and verify.

14.1 React Build Failed in Jenkins

Problem: The Jenkins pipeline failed during the React build stage.

Symptom: The build stopped on ESLint warnings; Jenkins runs with CI=true by default, and Create React App treats warnings as build-breaking errors in that mode.

Root Cause: The CI environment variable behaviour caused the React build to fail because warnings were treated as errors.

Diagnosis: The failure was traced to the Build React App stage in the Jenkins console output.

Fix:

```
bat 'set "CI=" && set "DISABLE_ESLINT_PLUGIN=true" && npm run build'
```

Verification: The pipeline was run again and the React application compiled successfully; the console showed "Compiled successfully."

Prevention: This exact build command was written into the Jenkinsfile's Build React App stage (Section 6.3) so the failure cannot recur.

Interview Explanation: My React build failed in Jenkins because Jenkins sets CI=true, and Create React App treats warnings as errors in that mode. I changed the build command to unset CI and disable the ESLint plugin, after which the build compiled successfully.

14.2 React Build Failed on AWS EC2

Problem: The React application build on EC2 failed.

Symptom: Environment key "jest/globals" is unknown.

Root Cause: The installed React/ESLint dependency combination caused an ESLint configuration compatibility problem during the build.

Diagnosis: Installed versions were checked with `npm list eslint eslint-config-react-app: react-scripts 5.0.1, eslint-config-react-app 7.0.1, eslint 8.57.1`.

Fix:

```
CI=false DISABLE_ESLINT_PLUGIN=true npm run build
```

Verification: The production build completed successfully after applying the fix.

Prevention: The build command was used after checking dependency versions, instead of forcing dependency upgrades that could introduce new issues.

Interview Explanation: The React build on EC2 failed because of an ESLint configuration compatibility issue. I checked the installed ESLint versions, then built the application with CI disabled and the ESLint plugin disabled, and the build completed successfully.

14.3 Jenkins SSH Authentication Failed

Problem: The Jenkins pipeline failed during the EC2 deployment stage.

Symptom: Permission denied (publickey,gssapi-keyex,...).

Root Cause: Jenkins was not using the correct private SSH key for connecting to the EC2 instance.

Diagnosis: The EC2 instance was reachable manually with the correct key pair while Jenkins could not authenticate, so the Jenkins credential ec2-ssh-key was inspected.

Fix:

```
# Correct EC2 private key configured under credential ID: ec2-ssh-key
# SSH username: ec2-user
```

Verification: The pipeline was run again; Jenkins connected to EC2, deployed the container, and finished with SUCCESS.

Prevention: The EC2 key pair and the Jenkins credential are checked before rerunning any deployment.

Interview Explanation: The Jenkins deployment initially failed because Jenkins was using an incorrect SSH credential. I replaced the credential with the correct EC2 key pair while keeping the same credential ID used in the Jenkinsfile, and Jenkins was then able to connect to EC2 and deploy successfully.

14.4 Docker Container Deployment Conflict

Problem: During redeployment, an existing container with the same name could prevent the new container from starting.

Root Cause: A previous container named react-container could still exist on the EC2 instance from an earlier deployment.

Diagnosis: The deployment stage of the Jenkinsfile was reviewed for container name conflicts.

Fix:

```
docker stop react-container || true
docker rm react-container || true
docker run -d -p 80:80 --name react-container \
  --restart unless-stopped akshayanimuthu/react-app:dev
```

Verification: The application became accessible through the EC2 address, with port 80 mapped to Nginx inside the container.

Prevention: The deployment pipeline now always stops and removes any existing container before starting the new version.

Interview Explanation: To make redeployment repeatable, the pipeline stops and removes the existing container before starting the new one. This prevents container-name conflicts on every redeploy.

14.5 Node Exporter Verification

Problem: Monitoring required system-level metrics from the EC2 server, so Node Exporter had to be confirmed working.

Root Cause: Node Exporter had to be installed, started, and exposing metrics on port 9100 before Prometheus could scrape it.

Diagnosis: The service was checked with `systemctl status node_exporter` and the metrics endpoint with `curl`.

Fix:

```
sudo systemctl daemon-reload
sudo systemctl enable --now node_exporter
curl http://localhost:9100/metrics | head
```

Verification: The service showed Active: active (running), and the metrics endpoint returned metric data.

Prevention: The service status and metrics endpoint are checked immediately after installation, before relying on it for monitoring.

Interview Explanation: I used Node Exporter to collect EC2 system metrics such as CPU, memory, disk and network usage. I verified the service status and checked the `/metrics` endpoint to confirm metrics were actually being exposed before configuring Prometheus.

14.6 Prometheus Target Verification

Problem: Prometheus needed to reliably collect metrics from Node Exporter.

Root Cause: The Node Exporter scrape target had to be configured correctly in `prometheus.yml`, and the Prometheus service had to be running.

Diagnosis: The configuration file was checked and validated using `promtool`.

Fix:

```
scrape_configs:
  - job_name: "node_exporter"
    static_configs:
      - targets: ["localhost:9100"]

sudo /opt/prometheus/promtool check config /etc/prometheus/prometheus.yml
```

Verification: Prometheus readiness returned "Prometheus Server is Ready.", and the Prometheus UI confirmed the `node_exporter` target as UP.

Prevention: The Prometheus configuration is validated with `promtool` before relying on the monitoring data it produces.

Interview Explanation: Prometheus was configured to scrape Node Exporter on localhost port 9100. I validated the configuration, checked Prometheus readiness, and confirmed the Node Exporter target was UP in the Prometheus UI.

14.7 Grafana SMTP Not Configured

Problem: Grafana displayed "SMTP not configured" when testing the email contact point.

Root Cause: The SMTP configuration lines in `grafana.ini` were commented out with a semicolon, so Grafana ignored them.

Diagnosis: The [smtp] section of grafana.ini was inspected and the settings were found to be commented out.

Fix:

```
[smtp]
enabled = true
host = smtp.gmail.com:587
user = YOUR_GMAIL_ADDRESS
password = YOUR_GOOGLE_APP_PASSWORD
from_address = YOUR_GMAIL_ADDRESS
from_name = Grafana
startTLS_policy = MandatoryStartTLS
```

Verification: After restarting Grafana it accepted the SMTP configuration and the email contact point test became available.

Prevention: SMTP settings must be uncommented and Grafana restarted after any change to grafana.ini.

Interview Explanation: Grafana initially reported that SMTP was not configured. I found the SMTP settings in grafana.ini were commented out, enabled them with Gmail SMTP and an App Password, restarted Grafana, and the email contact point test then succeeded.

14.8 Gmail SMTP Authentication Failed

Problem: Grafana's SMTP test failed with an authentication error.

Symptom: 535 5.7.8 Username and Password not accepted.

Root Cause: Gmail does not accept a normal account password for third-party SMTP logins.

Diagnosis: The Grafana SMTP test response identified the failure as an authentication error.

Fix:

```
# A Google App Password was generated and used as the SMTP password
# instead of the Gmail account password
# host = smtp.gmail.com:587, startTLS_policy = MandatoryStartTLS
```

Verification: The contact point test succeeded and a test email was received in Gmail.

Prevention: The App Password is stored only in Grafana's configuration and is never shared in documentation or screenshots.

Interview Explanation: Gmail rejected the SMTP login with error 535. I generated a Google App Password for Grafana and updated the SMTP password. After restarting Grafana, the contact point test succeeded and I received the email.

14.9 High CPU Alert Stayed in Normal State

Problem: The Grafana CPU alert stayed Normal because EC2 CPU usage never reached the configured threshold under normal conditions.

Root Cause: The CPU condition had not actually been met for the required evaluation period, so the rule had never been exercised.

Diagnosis: CPU usage was watched on the Grafana dashboard while load was generated on the EC2 instance.

Fix:

```
stress-ng --cpu 2 --timeout 5m
```

Verification: The alert changed to Firing, confirming the mechanism worked end-to-end; the final threshold was then restored to CPU above 80% with a 5-minute pending period.

Prevention: The alert was tested only for verification purposes, and the production threshold was restored immediately afterwards.

Interview Explanation: I tested the Grafana alert by generating CPU load on the EC2 instance with stress-ng. The alert changed to Firing, confirming the alerting mechanism worked, and I then restored the final threshold of 80% with a 5-minute pending period.

14.10 Verification Summary

Component	Final Status
GitHub repository	Working
Jenkins pipeline	SUCCESS
React build	Successful
Docker image	Built
Docker Hub image	Pushed
AWS EC2 deployment	Successful
React application	Accessible
Node Exporter	Running
Prometheus	Running, target UP
Grafana	Running, dashboard working
High CPU alert	Tested successfully
Gmail notification	Successfully received

14.11 Troubleshooting Commands Reference

```
docker ps
docker logs react-container
sudo systemctl status node_exporter
sudo systemctl status prometheus
curl http://localhost:9090/-/ready
curl http://localhost:9100/metrics | head
sudo systemctl status grafana-server
sudo systemctl restart grafana-server
```

15. Future Improvements

The current system satisfies the project's core objectives. The following are natural next steps toward a more production-grade setup:

- Add automated testing (unit/integration) as a pipeline stage before the build step.
- Add HTTPS via a custom domain name and an SSL/TLS certificate.
- Add container/application health checks so a failed deployment can be detected and rolled back automatically.
- Introduce separate staging and production branches with a promotion workflow between them.
- Externalize deployment configuration into environment variables rather than hard-coded values.
- Extend Grafana alerting to memory and disk usage thresholds in addition to CPU.
- Restrict the EC2 security group's SSH access (port 22) to specific trusted IP ranges.

16. Conclusion

This project implements a working, automated deployment pipeline for a React application: a GitHub commit on the dev branch flows through a Jenkins-driven React build, a Docker image build and push to Docker Hub, and an automated rollout to an AWS EC2 instance, with Node Exporter, Prometheus and Grafana providing continuous monitoring and a CPU alert that emails a notification through Gmail. Every stage described in this report was implemented and verified, as reflected in the checklist in Section 12.1.

Beyond the working deployment itself, the project — and the real problems worked through in the Troubleshooting section — demonstrates hands-on competency across the core DevOps toolchain: version control, containerization, CI/CD orchestration, cloud infrastructure and observability.

17. Project Screenshots

Each screenshot appears inline under its relevant section; this table is a reference index mapping figures to the source files in the repository's screenshots/ folder.

Figure	Filename	Shows	Section
4.1	architecture-diagram.png	System architecture / deployment and monitoring flow	4
4.2	github-repository-structure.png	Actual GitHub repository folder structure	4.4
6.1	jenkins-login-page.png	Jenkins sign-in page	6.2
6.2	jenkins-configuration-settings.png	Jenkins pipeline job configuration (SCM)	6.2
6.3	jenkins-credentials.png	Jenkins credentials store	6.5
6.4	jenkins-pipeline-execution.png	Successful Jenkins pipeline console output	6.6
7.1	aws-ec2-console.png	EC2 console showing the running instance	7.2
7.2	aws-security-group-configuration.png	EC2 security groups (ssh, http, grafana, node exporter, prometheus)	7.4
7.3	deployed-react-application.png	React application running via the EC2 public IP	7.7
8.1	dockerhub-repository-image-tags.png	Docker Hub repository showing the dev image tag	8.2
9.1	prometheus-target-up.png	Prometheus target health page (Node Exporter UP)	9.4
10.1	grafana-dashboard-1.png	Grafana dashboard — CPU usage / target status	10.2
10.2	grafana-dashboard-2.png	Grafana dashboard — network transmitted / received	10.2
11.1	grafana-high-cpu-alert.png	Grafana High CPU Usage alert rule (firing)	11.2
11.2	grafana-email-notification.png	Email notification received from the Grafana alert	11.4

18. Project Links

Resource	Location
GitHub Repository	Akshaya33-Devops/Automated-React-Deployment (dev branch)
Docker Hub Account	akshayanimuthu
Docker Image	akshayanimuthu/react-app:dev
Live Application	http://13.201.83.88
Architecture Diagram (source)	Architecture.png

Created for educational and portfolio purposes as part of DevOps training.